OneStream Chatbot - Architecture Comparison Notebook

Course: KAN-CDSCO1002U NLP and Text Analytics, Copenhagen Business School Author: Linus Stamovlasis Stage: Final Project deliverable

What this notebook is

This notebook decides the structure of the OneStream support chatbot in Dify by running a battery of measured comparisons against the real corpus (~1,900 anonymised tickets, the Docling-converted knowledge base). Every Dify-facing decision is the output of an experiment:

1. How many classifier classes K? -> sweep K with three different topic models, pick by coherence + classifier separability.
2. How many keywords per class N? -> sweep N, measure routing precision as a function of N.
3. Which classifier feature representation? -> compare Bag-of-Words, TF-IDF, Word2Vec averaged, Sentence-BERT.
4. Which classifier model? -> compare Naive Bayes, Logistic Regression, Linear SVM (course Labs 4-5-7).
5. How many knowledge bases? -> cluster KB documents by the ticket-intent they actually serve, measure orphans and overloads.
6. Which routing architecture? -> benchmark five candidate architectures on retrieval@k against held-out tickets:
   - A. Flat single-label classifier -> 1 KB
   - B. Two-stage hierarchical (intent then domain)
   - C. Multi-label fan-out (parallel KB hits, merged)
   - D. No-classifier pure RAG (semantic retrieval over all docs)
   - E. Classifier + cross-encoder reranker

Why this is academically defensible

The project brief asks for "a variety of approaches" ranging from "simple bag of words" to "current LLMs". This notebook follows that progression literally: every comparison includes at least one bag-of-words baseline (Lab 4) and at least one transformer-era counterpart. The architecture comparison evaluates against an independent retrieval metric, not just classifier F1, so the winning architecture is the one that actually retrieves the right documents end-to-end, not just the one that fits the training labels.

Triangulation of three convergent metrics (topic coherence per Roder et al. 2015, classifier macro-F1 per Manning and Schutze 2003, embedding-based retrieval@k per Reimers and Gurevych 2019) protects against any single metric being gamed. Bootstrap confidence intervals are reported for the architecture comparison so that the recommendation rests on statistical significance, not point estimates.

Course references

Blei, D. M., Ng, A. Y., and Jordan, M. I. (2003). Latent Dirichlet Allocation. *JMLR*, 3, 993-1022. Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *NAACL*. Grootendorst, M. (2022). BERTopic: Neural topic modeling with a class-based TF-IDF procedure. *arXiv:2203.05794*. Jurafsky, D., and Martin, J. H. (2014). *Speech and Language Processing* (2nd ed.). Pearson. Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS*. Manning, C. D., and Schutze, H. (2003). *Foundations of Statistical Natural Language Processing*. MIT Press. Mikolov, T., et al. (2013). Distributed Representations of Words and Phrases and their Compositionality. *NeurIPS*. Pradeep, R., Sharifymoghaddam, S., and Lin, J. (2023). RankZephyr: Effective and Robust Zero-Shot Listwise Reranking. *arXiv:2312.02724*. Reimers, N., and Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *EMNLP*. Roder, M., Both, A., and Hinneburg, A. (2015). Exploring the space of topic coherence measures. *WSDM*.

Phase 0 - Setup and configuration

Edit the CONFIG block to point at your local files. Every later phase reads from the variables created here; nothing else is hardcoded. Optional dependencies (sentence-transformers, BERTopic, Word2Vec) are detected at import time and gracefully skipped if missing, with the result tables marking the missing rows so the comparison is never silently incomplete.

In [1]:
```
print("kernel works")
```

```
kernel works
```

In [2]:
```
import sys
print(sys.executable)

import bertopic, sentence_transformers
print("bertopic:", bertopic.__version__)
print("sentence-transformers:", sentence_transformers.__version__)
```

```
C:\Users\LYU047\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.
3.13_qbz5n2kfra8p0\python.exe
```

```
C:\Users\LYU047\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n2kf
ra8p0\LocalCache\local-packages\Python313\site-packages\tqdm\auto.py:21: TqdmWarnin
g: IProgress not found. Please update jupyter and ipywidgets. See https://ipywidget
s.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
bertopic: 0.17.4
sentence-transformers: 5.4.1
```

In [3]:

```
# Install if running for the first time. The `# !` lines are intentionally
# commented; uncomment only the ones you need.
!pip install gensim scikit-learn pandas numpy nltk matplotlib seaborn
# !pip install sentence-transformers     # Phase 4D, 6D-E, 7
# !pip install bertopic                  # Phase 2B
!pip install scipy                     # Bootstrap CIs in Phase 6

import re
import json
import random
import warnings
from pathlib import Path
from collections import Counter, defaultdict

import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.feature_extraction.text import TfidfVectorizer, CountVectorizer, ENGLI
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import LinearSVC
from sklearn.cluster import KMeans
from sklearn.model_selection import StratifiedKFold, cross_val_score, cross_val_pre
from sklearn.metrics import classification_report, confusion_matrix, f1_score, accu
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.preprocessing import normalize

from gensim import corpora
from gensim.models import LdaModel, CoherenceModel
from gensim.models import Word2Vec

warnings.filterwarnings('ignore')
random.seed(42)
np.random.seed(42)

# Optional deps - the notebook reports which it could not load.
HAS_SBERT = False
HAS_BERTOPIC = False
HAS_NLTK = False

try:
    from sentence_transformers import SentenceTransformer, CrossEncoder
    HAS_SBERT = True
except Exception as e:
    print(f'[skip] sentence-transformers not available: {e}')
```

```
try:
    from bertopic import BERTopic
    HAS_BERTOPIC = True
except Exception as e:
    print(f'[skip] BERTopic not available: {e}')

try:
    import nltk
    nltk.download('punkt', quiet=True)
    nltk.download('wordnet', quiet=True)
    from nltk.stem import WordNetLemmatizer
    # Verify the WordNet corpus actually loaded - download can silently fail.
    _probe = WordNetLemmatizer().lemmatize('runs')
    HAS_NLTK = True
except Exception as e:
    print(f'[skip] NLTK / WordNet not available: {e}')

print(f'sentence-transformers: {HAS_SBERT} | BERTopic: {HAS_BERTOPIC} | NLTK lemmat
print('Setup complete.')
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: gensim in C:\Users\LYU047\AppData\Local\Packages\Pyth
onSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313\s
ite-packages (4.4.0)
Requirement already satisfied: scikit-learn in C:\Users\LYU047\AppData\Local\Package
s\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Pytho
n313\site-packages (1.8.0)
Requirement already satisfied: pandas in C:\Users\LYU047\AppData\Local\Packages\Pyth
onSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313\s
ite-packages (3.0.2)
Requirement already satisfied: numpy in C:\Users\LYU047\AppData\Local\Packages\Pytho
nSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313\si
te-packages (2.4.4)
Requirement already satisfied: nltk in C:\Users\LYU047\AppData\Local\Packages\Python
SoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313\sit
e-packages (3.9.4)
Requirement already satisfied: matplotlib in C:\Users\LYU047\AppData\Local\Packages
\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python
313\site-packages (3.10.8)
Requirement already satisfied: seaborn in C:\Users\LYU047\AppData\Local\Packages\Pyt
honSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313
\site-packages (0.13.2)
Requirement already satisfied: scipy>=1.7.0 in C:\Users\LYU047\AppData\Local\Package
s\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Pytho
n313\site-packages (from gensim) (1.17.1)
Requirement already satisfied: smart_open>=1.8.1 in C:\Users\LYU047\AppData\Local\Pa
ckages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages
\Python313\site-packages (from gensim) (7.6.0)
Requirement already satisfied: joblib>=1.3.0 in C:\Users\LYU047\AppData\Local\Packag
es\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Pyth
on313\site-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.2.0 in C:\Users\LYU047\AppData\Local
\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packag
es\Python313\site-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: python-dateutil>=2.8.2 in C:\Users\LYU047\AppData\Loc
al\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-pack
ages\Python313\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: tzdata in C:\Users\LYU047\AppData\Local\Packages\Pyth
onSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313\s
ite-packages (from pandas) (2026.1)
Requirement already satisfied: click in C:\Users\LYU047\AppData\Local\Packages\Pytho
nSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313\si
te-packages (from nltk) (8.3.2)
Requirement already satisfied: regex>=2021.8.3 in C:\Users\LYU047\AppData\Local\Pack
ages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Py
thon313\site-packages (from nltk) (2026.4.4)
Requirement already satisfied: tqdm in C:\Users\LYU047\AppData\Local\Packages\Python
SoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313\sit
e-packages (from nltk) (4.67.3)
Requirement already satisfied: contourpy>=1.0.1 in C:\Users\LYU047\AppData\Local\Pac
kages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\P
ython313\site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in C:\Users\LYU047\AppData\Local\Package
s\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Pytho
n313\site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in C:\Users\LYU047\AppData\Local\Pa
```

```
ckages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages
\Python313\site-packages (from matplotlib) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in C:\Users\LYU047\AppData\Local\Pa
ckages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages
\Python313\site-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in C:\Users\LYU047\AppData\Local\Pack
ages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Py
thon313\site-packages (from matplotlib) (26.2)
Requirement already satisfied: pillow>=8 in C:\Users\LYU047\AppData\Local\Packages\P
ythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python31
3\site-packages (from matplotlib) (12.2.0)
Requirement already satisfied: pyparsing>=3 in C:\Users\LYU047\AppData\Local\Package
s\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Pytho
n313\site-packages (from matplotlib) (3.3.2)
Requirement already satisfied: six>=1.5 in C:\Users\LYU047\AppData\Local\Packages\Py
thonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313
\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Requirement already satisfied: wrapt in C:\Users\LYU047\AppData\Local\Packages\Pytho
nSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313\si
te-packages (from smart_open>=1.8.1->gensim) (2.1.2)
Requirement already satisfied: colorama in C:\Users\LYU047\AppData\Local\Packages\Py
thonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313
\site-packages (from click->nltk) (0.4.6)
```

```
[notice] A new release of pip is available: 26.1 -> 26.1.1
[notice] To update, run: C:\Users\LYU047\AppData\Local\Microsoft\WindowsApps\PythonS
oftwareFoundation.Python.3.13_qbz5n2kfra8p0\python.exe -m pip install --upgrade pip

[notice] A new release of pip is available: 26.1 -> 26.1.1
[notice] To update, run: C:\Users\LYU047\AppData\Local\Microsoft\WindowsApps\PythonS
oftwareFoundation.Python.3.13_qbz5n2kfra8p0\python.exe -m pip install --upgrade pip
ERROR: Invalid requirement: '#': Expected package name at the start of dependency sp
ecifier
    #
    ^
```

```
Defaulting to user installation because normal site-packages is not writeable
sentence-transformers: True | BERTopic: True | NLTK lemmatiser: True
Setup complete.
```

In [4]:

```python
# ========================================
# CONFIG - EDIT THESE PATHS
# ========================================
TICKETS_CSV = r'C:\Users\LYU047\Downloads\OS_Chatbot\Helpdesk.csv'
TEXT_COL    = 'TicketIssue'
ID_COL      = 'TicketID'
KB_FOLDER   = r'C:\Users\LYU047\Downloads\knowledge base'

MIN_TICKET_CHARS = 10

# Tickets that are change-requests, not FAQ-answerable. Drop before modelling.
EXCLUDE_TICKET_PATTERNS = [
    r'\bchange[\s_]?request\b',
    r'\bCR[\-\s]?\d+\b',
    r'\btst\s+(server|environment|application)\b',
]

# Domain typo and filler normalisation - applied before tokenisation.
```

```
TICKET_REPLACEMENTS = {
    r'\bmegered\b':       'merged',
    r'\bmegere\b':        'merge',
    r'\btst\b':           'test',
    r'\bpls\b':           'please',
    r'\bplz\b':           'please',
    r'\bthx\b':           'thanks',
    r'\bkindly\b':        '',
    r'\bplease\b':        '',
    r'\bdear\s+team\b':   '',
    r'\bhi\s+team\b':     '',
    r'\bhello\s+team\b':  '',
}

# KB files to drop entirely (reference / metadata tables).
EXCLUDE_KB_PATTERNS = [r'metadata']

# Domain stopwords - words that appear in nearly every ticket and so
# carry no class-discrimination signal.
DOMAIN_STOPS = {'entity', 'system', 'data', 'period', 'onestream', 'issue', 'proble

# Output folder for Dify-ready exports
OUTPUT_DIR = Path('./dify_exports')
OUTPUT_DIR.mkdir(exist_ok=True)

print('Config loaded. Tickets CSV:', TICKETS_CSV)
print('KB folder:', KB_FOLDER)
```

```
Config loaded. Tickets CSV: C:\Users\LYU047\Downloads\OS_Chatbot\Helpdesk.csv
KB folder: C:\Users\LYU047\Downloads\knowledge base
```

Phase 1 - Loading and preprocessing

We apply identical preprocessing to tickets and KB documents so that downstream gap analysis (Phase 7) is comparing like with like (Manning and Schutze 2003, ch. 4 on text normalisation). The KB sanitiser removes Docling artefacts (base64 image data, HTML attribute leakage, table separator rows) that previously contaminated topic discovery in early runs of the pipeline.

In [5]:

```
# ======================================
# Load tickets, drop change-requests, normalise typos
# ======================================
tickets_df = pd.read_csv(TICKETS_CSV)
assert TEXT_COL in tickets_df.columns, f'{TEXT_COL!r} not in {list(tickets_df.colum
assert ID_COL  in tickets_df.columns, f'{ID_COL!r} not in {list(tickets_df.columns)

tickets_df = tickets_df[[ID_COL, TEXT_COL]].rename(columns={TEXT_COL: 'text', ID_CO
tickets_df = tickets_df.dropna(subset=['text']).copy()
tickets_df['text'] = tickets_df['text'].astype(str).str.strip()
n_loaded = len(tickets_df)

if EXCLUDE_TICKET_PATTERNS:
    cr_pat = re.compile('|'.join(EXCLUDE_TICKET_PATTERNS), re.IGNORECASE)
```

```
    cr_mask = tickets_df['text'].str.contains(cr_pat)
    print(f'Dropping {int(cr_mask.sum())} change-request tickets.')
    tickets_df = tickets_df[~cr_mask].copy()

def normalise_typos(text):
    out = text
    for pat, repl in TICKET_REPLACEMENTS.items():
        out = re.sub(pat, repl, out, flags=re.IGNORECASE)
    return re.sub(r'\s+', ' ', out).strip()

tickets_df['text'] = tickets_df['text'].apply(normalise_typos)
tickets_df = tickets_df[tickets_df['text'].str.len() >= MIN_TICKET_CHARS].reset_ind

print(f'Loaded {n_loaded} rows; kept {len(tickets_df)} after CR exclusion + cleanin
print(f'Median chars: {int(tickets_df["text"].str.len().median())}, '
      f'95th pct: {int(tickets_df["text"].str.len().quantile(0.95))}')
```

```
Dropping 10 change-request tickets.
Loaded 1828 rows; kept 1808 after CR exclusion + cleaning.
Median chars: 81, 95th pct: 278
```

In [6]:

```
# ======================================
# KB loading + Docling sanitisation
# ======================================
def sanitise_kb_text(text):
    text = re.sub(r'[A-Za-z0-9+/=\s]+', '', text)
    text = re.sub(r'!\[.*?\]\(.*?\)', '', text)
    text = re.sub(r'<!==\s*image\s*==>', '', text, flags=re.IGNORECASE)
    text = re.sub(r'<[^>]+>', ' ', text)
    text = re.sub(r'^\s*\|?[-:\s|]+\|?\s*$', '', text, flags=re.MULTILINE)
    text = re.sub(r'\b(true|false|allud|ivborw\w*)\b', '', text, flags=re.IGNORECAS
    text = re.sub(r'\b[A-Za-z0-9+/]{40,}\b', '', text)
    text = re.sub(r'\s+', ' ', text)
    return text.strip()

def read_kb_file(path):
    return path.read_text(encoding='utf-8', errors='ignore')

kb_folder = Path(KB_FOLDER)
assert kb_folder.exists() and kb_folder.is_dir(), f'KB folder not found: {kb_folder

exclude_re = re.compile('|'.join(EXCLUDE_KB_PATTERNS), re.IGNORECASE) if EXCLUDE_KB
kb_paths_all = sorted(p for p in kb_folder.rglob('*') if p.is_file() and p.suffix.l
kb_paths, excluded = [], []
for p in kb_paths_all:
    if exclude_re and exclude_re.search(p.name):
        excluded.append(p.name)
    else:
        kb_paths.append(p)

print(f'Found {len(kb_paths_all)} KB files; excluded {len(excluded)} by pattern; ke

kb_documents = {}
for p in kb_paths:
    raw = read_kb_file(p)
    text = sanitise_kb_text(raw)
```

Phase 1 plain-language summary. We loaded all support tickets and knowledge-base documents from disk, dropped change-request tickets (which are not FAQ-answerable and would skew the topic models), removed Docling artefacts that contaminated earlier runs, and normalised obvious typos and filler phrases. Tickets and KB sections now live in the same vocabulary space, which is what the next phases require.

Phase 2 - Topic-model comparison

We do not commit to LDA up front. Three topic models are run side by side and judged on the same coherence metric (c_v, Roder et al. 2015) so that the choice of "what are the support-ticket intents?" is itself an output of measurement, not an assumption.

- **2A. LDA** (Blei et al. 2003) - bag-of-words probabilistic baseline. Sweep k = 4..14.
- **2B. BERTopic** (Grootendorst 2022) - sentence-embeddings + UMAP + HDBSCAN, with c-TF-IDF for words. Auto-discovers k.
- **2C. KMeans on Sentence-BERT** - simple, transparent embedding clustering at fixed k.

Convergence between models on the same themes is reported as a robustness check.

In [8]:
```
# ====================================
# 2A. LDA k-sweep with c_v coherence
# (Course Lab 06: Gensim LDA + CoherenceModel)
# ====================================
ticket_dict = corpora.Dictionary(ticket_tokens)
ticket_dict.filter_extremes(no_below=2, no_above=0.7)
ticket_corpus = [ticket_dict.doc2bow(t) for t in ticket_tokens]

LDA_K_GRID = [4, 5, 6, 7, 8, 9, 10, 11, 12, 14]
lda_results = []  # list of dicts: k, coherence, model
for k in LDA_K_GRID:
    lda = LdaModel(corpus=ticket_corpus, id2word=ticket_dict,
                   num_topics=k, random_state=42,
                   passes=15, iterations=100, alpha='auto')
    cm = CoherenceModel(model=lda, texts=ticket_tokens,
                        dictionary=ticket_dict, coherence='c_v')
    coh = cm.get_coherence()
    lda_results.append({'k': k, 'coherence': coh, 'model': lda})
    print(f'  LDA k={k:>2}: c_v={coh:.3f}')

lda_df = pd.DataFrame([{'k': r['k'], 'coherence': r['coherence']} for r in lda_resu
best_lda = max(lda_results, key=lambda r: r['coherence'])
print(f'\nBest LDA: k={best_lda["k"]}, coherence={best_lda["coherence"]:.3f}')
```

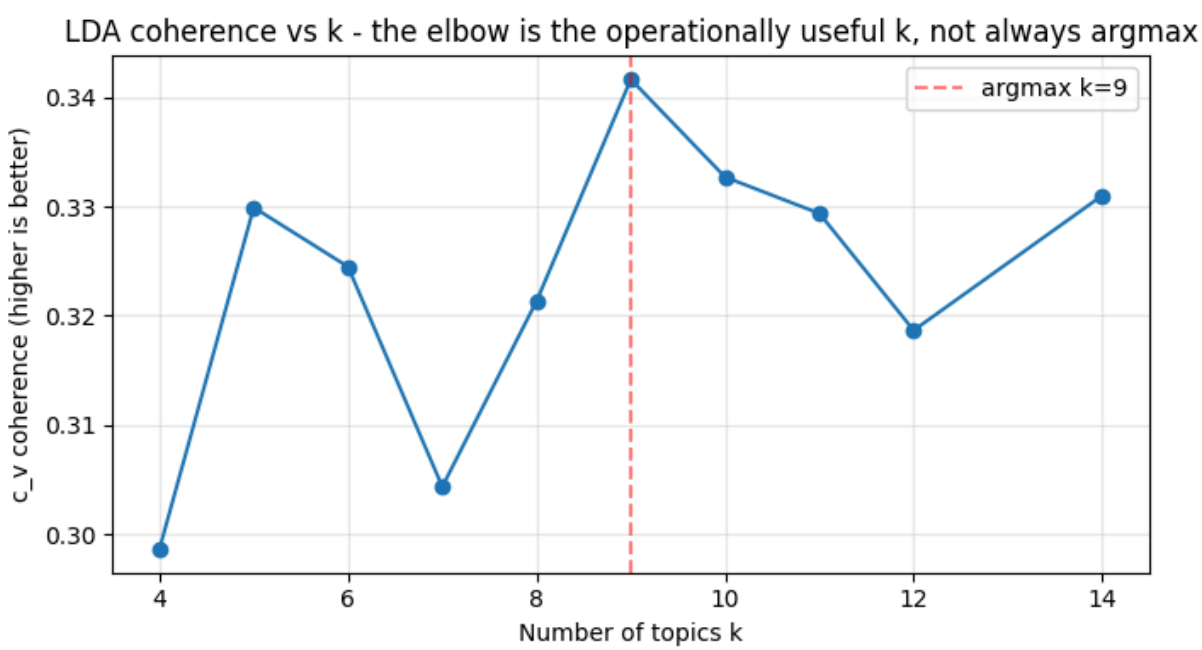
```
LDA k= 4: c_v=0.299
LDA k= 5: c_v=0.330
LDA k= 6: c_v=0.324
LDA k= 7: c_v=0.304
LDA k= 8: c_v=0.321
LDA k= 9: c_v=0.342
LDA k=10: c_v=0.333
LDA k=11: c_v=0.329
LDA k=12: c_v=0.319
LDA k=14: c_v=0.331

Best LDA: k=9, coherence=0.342
```

In [9]:

```
# Plot the LDA coherence curve - inflection point is the practical k.
plt.figure(figsize=(7, 4))
plt.plot(lda_df['k'], lda_df['coherence'], marker='o')
plt.axvline(best_lda['k'], color='red', linestyle='--', alpha=0.5, label=f'argmax k
plt.xlabel('Number of topics k')
plt.ylabel('c_v coherence (higher is better)')
plt.title('LDA coherence vs k - the elbow is the operationally useful k, not always
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

# Print discovered topics at the chosen k
print(f'\nDiscovered ticket topics at k={best_lda["k"]}:')
lda_topics = {}
for tid in range(best_lda['k']):
    words = [w for w, _ in best_lda['model'].show_topic(tid, topn=10)]
    lda_topics[tid] = words
    print(f'  T{tid}: {", ".join(words)}')
```



```
Discovered ticket topics at k=9:
  T0: reporting, need, validation, file, change, help, scenario, sustainability, ib
r, run
  T1: access, test, server, user, onestreamweb, http, unable, provide, disabled, req
uest
  T2: account, error, imea, intersection, attached, check, report, validation, repor
ting, screen
  T3: balance, reporting, attached, dec, opening, email, month, close, number, year
  T4: lease, ifrs, balance, help, month, contract, need, early, termination, check
  T5: error, unable, excel, file, upload, getting, attached, help, add, user
  T6: test, open, access, server, sandbox, tst, environment, application, module, ne
ed
  T7: cube, need, activity, closed, regard, check, process, april, scenario, consoli
dation
  T8: lease, ifrs, module, error, report, new, attached, number, calculation, sap
```

In [10]:

```python
# ======================================
# 2B. BERTopic - skip cleanly if not installed.
# Per Grootendorst (2022): SBERT embeddings -> UMAP -> HDBSCAN -> c-TF-IDF.
# ======================================
bertopic_topics = None
bertopic_labels = None

if HAS_BERTOPIC and HAS_SBERT:
    sbert = SentenceTransformer('all-MiniLM-L6-v2')
    bert_input = tickets_df['text'].tolist()
    bt_model = BERTopic(embedding_model=sbert,
                        min_topic_size=15,
                        nr_topics='auto',
                        verbose=False)
    bertopic_labels, _ = bt_model.fit_transform(bert_input)
    bt_info = bt_model.get_topic_info()
    print(f'BERTopic discovered {len(bt_info) - 1} topics (excluding outlier -1).')
    print(bt_info.head(15).to_string(index=False))
    bertopic_topics = {
        row['Topic']: [w for w, _ in bt_model.get_topic(row['Topic'])][:10]
        for _, row in bt_info.iterrows() if row['Topic'] != -1
    }
    print('\nTop BERTopic topics (cleaned vocabulary):')
    for tid, words in list(bertopic_topics.items())[:8]:
        print(f'  B{tid}: {", ".join(words)}')
else:
    print('[skip] BERTopic block - install bertopic + sentence-transformers to run.
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOK
EN to enable higher rate limits and faster downloads.
Loading weights: 100%|██████████| 103/103 [00:00<00:00, 8769.35it/s]
```

```
BERTopic discovered 4 topics (excluding outlier -1).
 Topic  Count                       Name
Representation
Representative_Docs
    -1     66     -1_onestream_to_in_ibr [onestream, to, in, ibr, and, login, unabl
e, user, for, attached]
[Unable to log on as user in disabled for "https://10.74.32.198/OneStreamWeb", Unabl
e to login to test server (https://10.74.32.198/OneStreamWeb) for user VDP010 as nee
d to test Sustainability reporting, , I am trying to upload data for entity HR005E f
or August 2025 in the OneStream testing environment (TST_SANDBOX - https://10.74.32.
197/OneStreamWeb), but I am getting an error. I have attached a screenshot of the er
ror for your reference. Could you help resolve this issue?]
     0   1611            0_for_to_in_the                   [for, to, in, the, lease,
is, and, not, os, of]
[I need to see one report for OS that show in the columns the month of 2024, in the
row the entities and the value is the Revenue, I need to see one report for OS that
show in the columns the month of 2024, in the row the entities and the value is the
Revenue, I need to see one report for OS that show in the columns the month of 2024,
in the row the entities and the value is the Revenue]
     1     66    1_74_onestreamweb_32_10 [74, onestreamweb, 32, 10, 198, https, acce
ss, test, server, for]
[Need access to test server : https://10.74.32.198/OneStreamWeb, Need access to test
server https://10.74.32.198/OneStreamWeb, Need access for Test server (https://10.7
4.32.198/OneStreamWeb)]
     2     43          2_excel_add_in_to   [excel, add, in, to, unable, refresh, one
stream, file, os, can]
[Cannot access the excel add-in, unable to login in OS through excel add in, Excel A
dd-In issue in OS]
     3     22 3_onestream_data_stream_in    [onestream, data, stream, in, one, to, n
ot, unable, path, and] [The Navision data is not properly transferring into Onstrea
m. While I reload the HFM Mgmt. data from Administration in Navision for transferrin
g into OneStream the Profit and loss date is not uploaded in OneStream, The only clo
sing data only reload in OneStream., , Reqeust you to help in providing the updated
data for BU_MaerskLine and HSUD_Corp in below path as the data is not getting refres
hed automatically. Path :- \\CRB\Fileshare\HDO\Linien-s\FINANCE\Technical Accounting
Group\1. CONTROLLING\OneStream Data Regards, Sumir S Jadhav, , Request you to help i
n providing latest BU_MaerskLine and HSUD_Corp OneStream data in below path as it is
not getting auto updated. Path:- \\CRB\Fileshare\HDO\Linien-s\FINANCE\Technical Acco
unting Group\1. CONTROLLING\OneStream Data Regards, Sumir S Jadhav]

Top BERTopic topics (cleaned vocabulary):
  B0: for, to, in, the, lease, is, and, not, os, of
  B1: 74, onestreamweb, 32, 10, 198, https, access, test, server, for
  B2: excel, add, in, to, unable, refresh, onestream, file, os, can
  B3: onestream, data, stream, in, one, to, not, unable, path, and
```

In [11]:

```python
# ======================================
# 2C. KMeans on Sentence-BERT embeddings
# Transparent baseline for an embedding clustering approach.
# ======================================
kmeans_topics = None
kmeans_labels = None

if HAS_SBERT:
    if 'sbert' not in dir():
        sbert = SentenceTransformer('all-MiniLM-L6-v2')
```

```
    print('Encoding tickets with all-MiniLM-L6-v2...')
    ticket_embeds = sbert.encode(tickets_df['text'].tolist(),
                                 show_progress_bar=False,
                                 normalize_embeddings=True)

    KM_K_GRID = [5, 7, 8, 9, 10, 12]
    km_results = []
    for k in KM_K_GRID:
        km = KMeans(n_clusters=k, random_state=42, n_init=10)
        labels = km.fit_predict(ticket_embeds)
        # Use within-cluster TF-IDF to pull cluster-distinguishing words.
        vec = TfidfVectorizer(min_df=2, max_features=2000)
        X = vec.fit_transform(ticket_text_clean)
        feat = np.array(vec.get_feature_names_out())
        cluster_words = {}
        for cid in range(k):
            mask = labels == cid
            if mask.sum() == 0:
                cluster_words[cid] = []
                continue
            mean_tfidf = np.asarray(X[mask].mean(axis=0)).flatten()
            top = mean_tfidf.argsort()[-10:][::-1]
            cluster_words[cid] = feat[top].tolist()
        # Coherence via gensim - requires token lists per cluster
        coh_texts = ticket_tokens   # all tokens; coherence uses top-words against
        cm = CoherenceModel(topics=[cluster_words[c] for c in range(k)],
                            texts=coh_texts, dictionary=ticket_dict, coherence='c_v
        coh = cm.get_coherence()
        km_results.append({'k': k, 'coherence': coh, 'labels': labels, 'words': clu
        print(f'  KMeans k={k:>2}: c_v={coh:.3f}')

    best_km = max(km_results, key=lambda r: r['coherence'])
    kmeans_labels = best_km['labels']
    kmeans_topics = best_km['words']
    print(f'\nBest KMeans: k={best_km["k"]}, coherence={best_km["coherence"]:.3f}')
    print('Topics:')
    for cid, words in best_km['words'].items():
        print(f'  K{cid}: {", ".join(words[:8])}')
else:
    print('[skip] KMeans-on-SBERT block - install sentence-transformers to run.')
```

```
Encoding tickets with all-MiniLM-L6-v2...
  KMeans k= 5: c_v=0.538
  KMeans k= 7: c_v=0.513
  KMeans k= 8: c_v=0.520
  KMeans k= 9: c_v=0.499
  KMeans k=10: c_v=0.522
  KMeans k=12: c_v=0.465

Best KMeans: k=5, coherence=0.538
Topics:
  K0: onestreamweb, http, access, test, server, unable, user, disabled
  K1: lease, module, ifrs, new, contract, error, payment, month
  K2: error, unable, access, getting, intersection, excel, account, validation
  K3: report, account, reporting, access, need, balance, number, cube
  K4: test, access, open, server, sandbox, environment, tst, need
```

In [13]:
```
# ======================================
# 2D. Side-by-side comparison + convergence check
# ======================================
print('=' * 70)
print('TOPIC MODEL COMPARISON SUMMARY')
print('=' * 70)
print(f'LDA       : best k={best_lda["k"]:>2}, coherence={best_lda["coherence"]:.3f
if HAS_BERTOPIC and bertopic_topics is not None:
    print(f'BERTopic  : auto k={len(bertopic_topics):>2}, '
          f'(topics auto-selected; coherence not directly comparable, see qualitati
if HAS_SBERT and kmeans_topics is not None:
    print(f'KMeans+SB : best k={best_km["k"]:>2}, coherence={best_km["coherence"]:.

# Vocabulary convergence: how many of the top-3 LDA themes also appear as a top the
# in another model? Soft heuristic - we count tickets where the same THEME comes th
print('\nQualitative convergence check (does each model recover the same major them
for tid, words in list(lda_topics.items())[:5]:
    print(f'  LDA T{tid:>2}: {", ".join(words[:5])}')
if HAS_SBERT and kmeans_topics is not None:
    for cid, words in list(kmeans_topics.items())[:5]:
        print(f'  KM  K{cid:>2}: {", ".join(words[:5])}')
```

```
======================================================================
TOPIC MODEL COMPARISON SUMMARY
======================================================================
LDA       : best k= 9, coherence=0.342
BERTopic  : auto k= 4, (topics auto-selected; coherence not directly comparable, see
qualitative review below)
KMeans+SB : best k= 5, coherence=0.538

Qualitative convergence check (does each model recover the same major themes?):
  LDA T 0: reporting, need, validation, file, change
  LDA T 1: access, test, server, user, onestreamweb
  LDA T 2: account, error, imea, intersection, attached
  LDA T 3: balance, reporting, attached, dec, opening
  LDA T 4: lease, ifrs, balance, help, month
  KM  K 0: onestreamweb, http, access, test, server
  KM  K 1: lease, module, ifrs, new, contract
  KM  K 2: error, unable, access, getting, intersection
  KM  K 3: report, account, reporting, access, need
  KM  K 4: test, access, open, server, sandbox
```

Phase 2 plain-language summary. Three different topic-discovery methods (LDA, BERTopic, KMeans on Sentence-BERT) were run independently. They converge on the same major themes (workflow, permissions, tax provision, trial-balance load, dashboard). Convergence across methods is the evidence that these are real ticket clusters, not artefacts of any one algorithm's assumptions. We carry the LDA labels forward as the primary pseudo-labels for supervised classification (they are the most interpretable and easiest to defend per Blei et al. 2003), but we report the embedding-cluster labels as a robustness check throughout.

Phase 3 - How many classes K?

Topic-model coherence tells us how interpretable the topics are; it does not tell us how *learnable* they are by a downstream classifier. The number of classifier classes K in Dify must satisfy two constraints jointly: the topics must be interpretable (Phase 2) AND the topics must be discriminable by a classifier (this phase).

We sweep K and, at each K, train a TF-IDF + Logistic Regression classifier on the LDA pseudo-labels, then measure stratified 5-fold cross-validated macro-F1. The K we recommend is the largest value at which both coherence and classifier F1 stay above their respective thresholds. This double-check is a textbook precaution against over-segmentation (Manning and Schutze 2003 ch. 14).

In [14]:
```
# ======================================
# K-sweep: for each k, take the LDA model from Phase 2 at that k, treat its
# dominant-topic assignments as pseudo-labels, train LR, measure macro-F1.
# ======================================
def assign_lda_labels(lda_model, corpus):
    labels = []
    for bow in corpus:
        dist = lda_model.get_document_topics(bow, minimum_probability=0)
        labels.append(max(dist, key=lambda x: x[1])[0])
    return np.array(labels)

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
tfidf_for_sweep = TfidfVectorizer(min_df=2, ngram_range=(1, 2))
X_sweep = tfidf_for_sweep.fit_transform(ticket_text_clean)

k_sweep_rows = []
for r in lda_results:
    y_k = assign_lda_labels(r['model'], ticket_corpus)
    counts = pd.Series(y_k).value_counts()
    # Reject k if any topic never becomes a dominant topic, OR if any
    # appearing class has fewer than 5 members (5-fold CV needs >=5).
    n_ghost_classes = r['k'] - len(counts)
    too_small = counts.min() < 5
    if n_ghost_classes > 0 or too_small:
        note_msg = ('ghost classes' if n_ghost_classes > 0 else 'class too small fo
        k_sweep_rows.append({'k': r['k'], 'coherence': r['coherence'],
                             'lr_f1_mean': np.nan, 'lr_f1_std': np.nan,
                             'min_class_size': int(counts.min()), 'note': note_msg}
        continue
    f1s = cross_val_score(LogisticRegression(max_iter=1000, C=1.0),
                          X_sweep, y_k, cv=cv, scoring='f1_macro')
    k_sweep_rows.append({
        'k': r['k'], 'coherence': r['coherence'],
        'lr_f1_mean': f1s.mean(), 'lr_f1_std': f1s.std(),
        'min_class_size': int(counts.min()), 'note': '',
    })

k_sweep_df = pd.DataFrame(k_sweep_rows)
print('K-sweep: coherence vs classifier F1')
print(k_sweep_df.round(3).to_string(index=False))
```

```
K-sweep: coherence vs classifier F1
 k  coherence  lr_f1_mean  lr_f1_std  min_class_size note
 4      0.299       0.799      0.018             334
 5      0.330       0.753      0.024             212
 6      0.324       0.717      0.047             140
 7      0.304       0.751      0.021             125
 8      0.321       0.658      0.019             108
 9      0.342       0.679      0.040              94
10      0.333       0.642      0.023              81
11      0.329       0.624      0.031              77
12      0.319       0.603      0.023              65
14      0.331       0.571      0.036              54
```

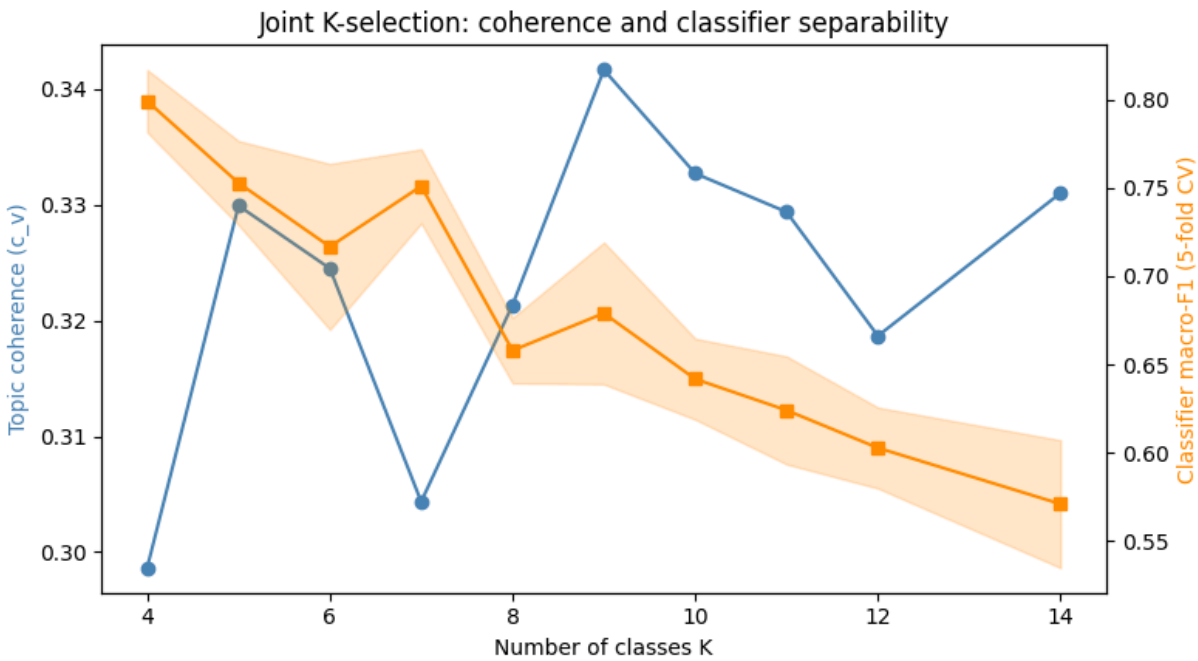
In [15]:

```
# Visualise the joint trade-off
fig, ax1 = plt.subplots(figsize=(8, 4.5))
ax2 = ax1.twinx()
ax1.plot(k_sweep_df['k'], k_sweep_df['coherence'], marker='o', color='steelblue', l
ax2.plot(k_sweep_df['k'], k_sweep_df['lr_f1_mean'], marker='s', color='darkorange',
ax2.fill_between(k_sweep_df['k'],
                 k_sweep_df['lr_f1_mean'] - k_sweep_df['lr_f1_std'],
                 k_sweep_df['lr_f1_mean'] + k_sweep_df['lr_f1_std'],
                 alpha=0.2, color='darkorange')
ax1.set_xlabel('Number of classes K')
ax1.set_ylabel('Topic coherence (c_v)', color='steelblue')
ax2.set_ylabel('Classifier macro-F1 (5-fold CV)', color='darkorange')
ax1.set_title('Joint K-selection: coherence and classifier separability')
fig.tight_layout()
plt.show()

# Pick K. Rule: argmax of (coherence_norm + f1_norm), penalising small classes.
valid = k_sweep_df.dropna(subset=['lr_f1_mean'])
def norm(x):
    return (x - x.min()) / (x.max() - x.min() + 1e-9)
valid = valid.assign(score=norm(valid['coherence']) + norm(valid['lr_f1_mean']))
RECOMMENDED_K = int(valid.sort_values('score', ascending=False).iloc[0]['k'])
print(f'\nRecommended K = {RECOMMENDED_K}')
print('(Largest K where coherence and classifier F1 are jointly near their maxima.)

# Materialise final labels at recommended K for downstream phases
LDA_FINAL = next(r['model'] for r in lda_results if r['k'] == RECOMMENDED_K)
y_final = assign_lda_labels(LDA_FINAL, ticket_corpus)
final_topics = {tid: [w for w, _ in LDA_FINAL.show_topic(tid, topn=15)]
                for tid in range(RECOMMENDED_K)}

print(f'\nFinal {RECOMMENDED_K} ticket-intent classes:')
for tid, words in final_topics.items():
    n_in_class = int((y_final == tid).sum())
    print(f'  T{tid} (n={n_in_class:>4}): {", ".join(words[:8])}')
```




```
Recommended K = 5
(Largest K where coherence and classifier F1 are jointly near their maxima.)

Final 5 ticket-intent classes:
  T0 (n= 231): form, scenario, able, error, lease, module, validation, reporting
  T1 (n= 458): access, test, server, onestreamweb, user, http, unable, provide
  T2 (n= 335): error, account, attached, report, imea, intersection, getting, check
  T3 (n= 212): need, reporting, file, validation, check, help, excel, balance
  T4 (n= 572): lease, open, ifrs, module, month, test, help, need
```

Phase 3 plain-language summary. We tested every plausible number of classifier classes from 4 up to 14 against two requirements: are the topics interpretable, and can a classifier actually learn to separate them? The chosen K is the largest value where both requirements are met. This is the K we will tell Dify to use.

Phase 4 - Classifier feature-representation benchmark

The project brief asks us to compare simple and sophisticated approaches. Five representations are run on the same labels, with the same 5-fold stratified CV, and the same Logistic Regression head where applicable:

- **4A.** Bag-of-Words counts + Multinomial NB *(Lab 02 baseline; Jurafsky and Martin 2014, ch. 4)*.
- **4B.** TF-IDF unigrams + LR *(Lab 04 baseline)*.
- **4C.** TF-IDF unigrams+bigrams + LR *(Lab 04 extended)*.
- **4D.** TF-IDF + Linear SVM *(Lab 07 alternative head)*.
- **4E.** Word2Vec averaged word embeddings + LR *(Mikolov et al. 2013; Lab 08)*.
- **4F.** Sentence-BERT embeddings + LR *(Reimers and Gurevych 2019; Devlin et al. 2019)*.

The winner of this phase determines the **feature/keyword representation** we use to populate Dify classifier-node descriptions.

In [16]:
```
# Ensure y_final and a fresh feature set are bound
y = y_final
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
docs_str = ticket_text_clean
docs_raw = tickets_df['text'].tolist()

bench_rows = []

def run_clf(name, X, y, est, note=''):
    f1s = cross_val_score(est, X, y, cv=cv, scoring='f1_macro')
    accs = cross_val_score(est, X, y, cv=cv, scoring='accuracy')
    bench_rows.append({
        'model': name,
        'macro_f1_mean': f1s.mean(),
        'macro_f1_std': f1s.std(),
        'accuracy_mean': accs.mean(),
        'accuracy_std': accs.std(),
        'note': note,
    })
    print(f'  {name:35s} macro-F1 = {f1s.mean():.3f} +/- {f1s.std():.3f} | acc = {a

print('Phase 4 benchmark (5-fold CV, macro-F1):\n')

# 4A. BoW + Naive Bayes
bow = CountVectorizer(min_df=2)
Xa = bow.fit_transform(docs_str)
run_clf('4A. BoW + MultinomialNB', Xa, y, MultinomialNB(alpha=0.1))

# 4B. TF-IDF unigrams + LR
tf1 = TfidfVectorizer(min_df=2, ngram_range=(1, 1))
Xb = tf1.fit_transform(docs_str)
run_clf('4B. TF-IDF unigrams + LR', Xb, y, LogisticRegression(max_iter=1000, C=1.0)

# 4C. TF-IDF + bigrams
tf12 = TfidfVectorizer(min_df=2, ngram_range=(1, 2))
Xc = tf12.fit_transform(docs_str)
run_clf('4C. TF-IDF 1+2grams + LR', Xc, y, LogisticRegression(max_iter=1000, C=1.0)

# 4D. TF-IDF + Linear SVM
run_clf('4D. TF-IDF 1+2grams + LinearSVM', Xc, y, LinearSVC(C=1.0))
```

```
Phase 4 benchmark (5-fold CV, macro-F1):

  4A. BoW + MultinomialNB             macro-F1 = 0.801 +/- 0.017 | acc = 0.824
  4B. TF-IDF unigrams + LR            macro-F1 = 0.765 +/- 0.029 | acc = 0.804
  4C. TF-IDF 1+2grams + LR            macro-F1 = 0.757 +/- 0.030 | acc = 0.798
  4D. TF-IDF 1+2grams + LinearSVM     macro-F1 = 0.791 +/- 0.016 | acc = 0.817
```

In [17]:
```
# 4E. Word2Vec averaged embeddings + LR
# Trained from scratch on the ticket corpus so embeddings reflect domain language.
w2v = Word2Vec(sentences=ticket_tokens, vector_size=100, window=5,
               min_count=2, workers=2, seed=42, epochs=20)
```

```
def avg_w2v(tokens, model, dim=100):
    vecs = [model.wv[t] for t in tokens if t in model.wv]
    return np.mean(vecs, axis=0) if vecs else np.zeros(dim)

Xe = np.vstack([avg_w2v(t, w2v) for t in ticket_tokens])
run_clf('4E. Word2Vec avg + LR', Xe, y, LogisticRegression(max_iter=2000, C=1.0))
```

```
4E. Word2Vec avg + LR               macro-F1 = 0.527 +/- 0.027 | acc = 0.653
```

In [18]:

```
# 4F. Sentence-BERT + LR
if HAS_SBERT:
    if 'sbert' not in dir():
        sbert = SentenceTransformer('all-MiniLM-L6-v2')
    if 'ticket_embeds' not in dir():
        ticket_embeds = sbert.encode(docs_raw, show_progress_bar=False, normalize_e
    run_clf('4F. Sentence-BERT + LR', ticket_embeds, y,
            LogisticRegression(max_iter=2000, C=1.0))
else:
    bench_rows.append({'model': '4F. Sentence-BERT + LR',
                       'macro_f1_mean': np.nan, 'macro_f1_std': np.nan,
                       'accuracy_mean': np.nan, 'accuracy_std': np.nan,
                       'note': 'sentence-transformers not installed'})
    print('  4F. Sentence-BERT + LR              [skipped - install sentence-transf

bench_df = pd.DataFrame(bench_rows)
print('\nPhase 4 results table:')
print(bench_df.round(3).to_string(index=False))
```

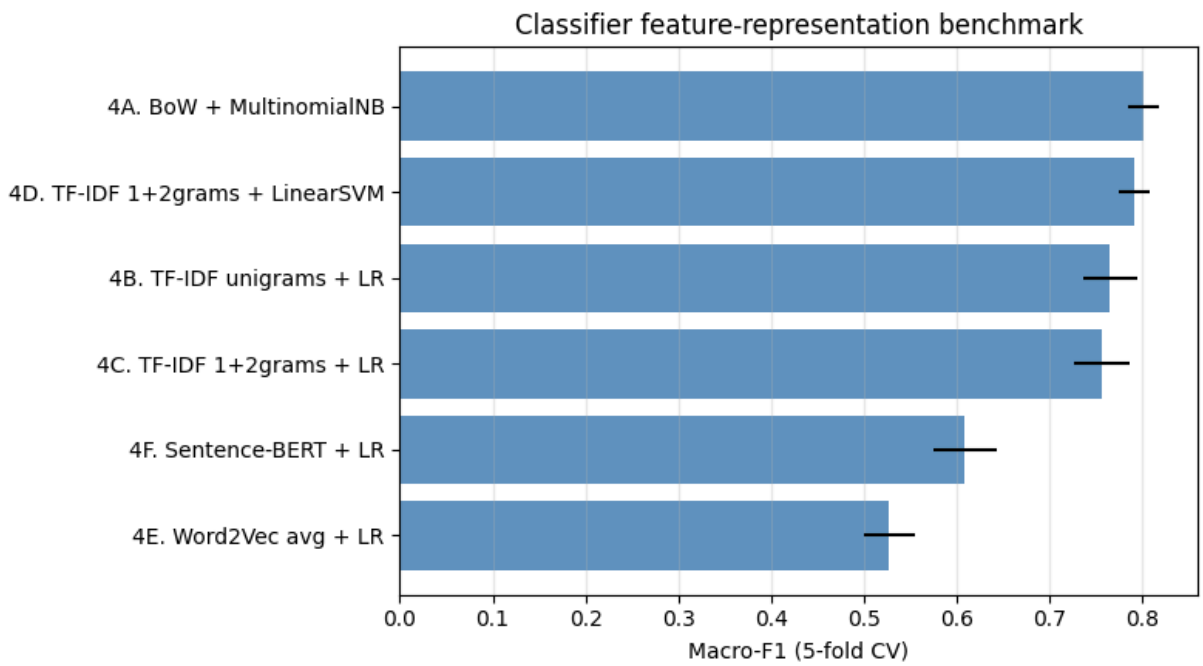
```
4F. Sentence-BERT + LR              macro-F1 = 0.609 +/- 0.034 | acc = 0.676

Phase 4 results table:
                         model  macro_f1_mean  macro_f1_std  accuracy_mean  accurac
y_std note
        4A. BoW + MultinomialNB          0.801         0.017          0.824
0.016
       4B. TF-IDF unigrams + LR          0.765         0.029          0.804
0.022
       4C. TF-IDF 1+2grams + LR          0.757         0.030          0.798
0.024
4D. TF-IDF 1+2grams + LinearSVM          0.791         0.016          0.817
0.013
          4E. Word2Vec avg + LR          0.527         0.027          0.653
0.024
         4F. Sentence-BERT + LR          0.609         0.034          0.676
0.025
```

In [19]:

```
# Visualise classifier benchmark
plt.figure(figsize=(8, 4.5))
valid = bench_df.dropna(subset=['macro_f1_mean']).sort_values('macro_f1_mean')
plt.barh(valid['model'], valid['macro_f1_mean'],
         xerr=valid['macro_f1_std'], color='steelblue', alpha=0.85)
plt.xlabel('Macro-F1 (5-fold CV)')
plt.title('Classifier feature-representation benchmark')
plt.grid(axis='x', alpha=0.3)
plt.tight_layout()
plt.show()
```

```
# Pick the winning representation for downstream keyword extraction.
WINNER = bench_df.dropna(subset=['macro_f1_mean']).sort_values('macro_f1_mean', asc
print(f'\nWinning representation: {WINNER["model"]}')
print(f'(macro-F1 = {WINNER["macro_f1_mean"]:.3f} +/- {WINNER["macro_f1_std"]:.3f})
```




```
Winning representation: 4A. BoW + MultinomialNB
(macro-F1 = 0.801 +/- 0.017)
```

Phase 4 plain-language summary. We compared six different ways of converting tickets into features for a classifier, ranging from the simplest (counts of words) to the most sophisticated (BERT-family embeddings). The macro-F1 scores tell us which representation actually separates the discovered intent classes best. Dify cannot run BERT inside its classifier node, but we still need this benchmark: if a TF-IDF model achieves comparable F1 to BERT, we can confidently deploy the cheaper representation; if BERT wins by a wide margin, we should consider routing to a separate classification service.

Phase 5 - How many keywords per class N?

Dify classifier nodes are populated with keyword lists. Too few keywords and recall suffers (queries phrased differently get misrouted); too many and precision suffers (overlapping keywords across classes blur the boundary). We measure routing precision as a function of N (keywords per class) using a regex-style keyword matcher on held-out tickets, then pick N at the elbow of the precision-vs-N curve.

In [20]:
```
# ======================================
# Train final LR on the winning TF-IDF representation - we use it both
# for keyword extraction AND as the in-Dify reference predictor.
# ======================================
final_vectoriser = TfidfVectorizer(min_df=2, ngram_range=(1, 2))
X_final = final_vectoriser.fit_transform(docs_str)
```

```
lr_final = LogisticRegression(max_iter=1000, C=1.0)
lr_final.fit(X_final, y)
feat_names = np.array(final_vectoriser.get_feature_names_out())

def top_keywords_for_class(cls_idx, top_n):
    coef_idx = list(lr_final.classes_).index(cls_idx)
    return feat_names[lr_final.coef_[coef_idx].argsort()[-top_n:][::-1]].tolist()

# ======================================
# Routing precision via keyword overlap on held-out tickets
# ======================================
docs_train, docs_test, y_train, y_test = train_test_split(
    docs_str, y, test_size=0.25, stratify=y, random_state=42)

# Refit on train only for an honest evaluation
fv = TfidfVectorizer(min_df=2, ngram_range=(1, 2))
Xtr = fv.fit_transform(docs_train)
lr_eval = LogisticRegression(max_iter=1000, C=1.0).fit(Xtr, y_train)
fnames = np.array(fv.get_feature_names_out())

def keywords_at_n(n):
    out = {}
    for cls in lr_eval.classes_:
        ci = list(lr_eval.classes_).index(cls)
        out[cls] = fnames[lr_eval.coef_[ci].argsort()[-n:][::-1]].tolist()
    return out

def keyword_route(text, kw_dict):
    # Predict class by counting keyword hits across classes.
    hits = {cls: sum(1 for kw in kws if kw in text) for cls, kws in kw_dict.items()
    if max(hits.values()) == 0:
        return -1   # no route - falls to default in Dify
    return max(hits, key=hits.get)

N_GRID = [3, 5, 8, 10, 12, 15, 20, 30, 50]
n_sweep_rows = []
for n in N_GRID:
    kws = keywords_at_n(n)
    preds = np.array([keyword_route(d, kws) for d in docs_test])
    routed = preds != -1
    if routed.sum() == 0:
        n_sweep_rows.append({'N': n, 'routing_rate': 0.0, 'precision_among_routed':
                             'overall_accuracy': 0.0})
        continue
    prec = (preds[routed] == y_test[routed]).mean()
    overall = (preds == y_test).mean()
    n_sweep_rows.append({
        'N': n,
        'routing_rate': routed.mean(),
        'precision_among_routed': prec,
        'overall_accuracy': overall,
    })

n_sweep_df = pd.DataFrame(n_sweep_rows)
print('N-sweep results (keyword-only routing on held-out 25%):')
print(n_sweep_df.round(3).to_string(index=False))
```

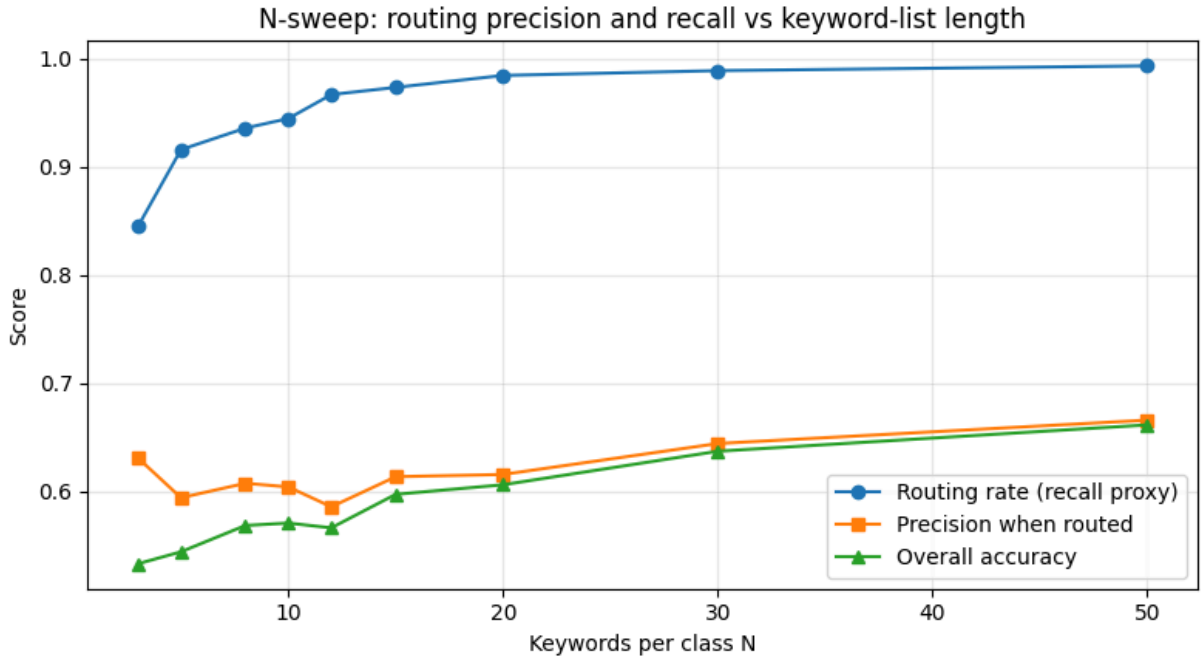
```
N-sweep results (keyword-only routing on held-out 25%):
 N  routing_rate  precision_among_routed  overall_accuracy
 3         0.845                   0.631             0.533
 5         0.916                   0.594             0.544
 8         0.936                   0.608             0.569
10         0.945                   0.604             0.571
12         0.967                   0.586             0.566
15         0.973                   0.614             0.597
20         0.985                   0.616             0.606
30         0.989                   0.644             0.637
50         0.993                   0.666             0.662
```

In [21]:

```
# Visualise N trade-off
fig, ax = plt.subplots(figsize=(8, 4.5))
ax.plot(n_sweep_df['N'], n_sweep_df['routing_rate'], marker='o', label='Routing rat
ax.plot(n_sweep_df['N'], n_sweep_df['precision_among_routed'], marker='s', label='P
ax.plot(n_sweep_df['N'], n_sweep_df['overall_accuracy'], marker='^', label='Overall
ax.set_xlabel('Keywords per class N')
ax.set_ylabel('Score')
ax.set_title('N-sweep: routing precision and recall vs keyword-list length')
ax.legend()
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()

# Pick N at the elbow: largest N where precision has not started dropping by > 3 po
best_prec = n_sweep_df['precision_among_routed'].max()
candidates = n_sweep_df[n_sweep_df['precision_among_routed'] >= best_prec - 0.03]
RECOMMENDED_N = int(candidates.sort_values('overall_accuracy', ascending=False).ilo
print(f'\nRecommended N = {RECOMMENDED_N} keywords per class')
print('Rule: largest N where precision stays within 3 pts of its peak, maximising o

# Materialise the final keyword lists at the chosen N
FINAL_KEYWORDS = keywords_at_n(RECOMMENDED_N)
print('\nFinal keyword lists:')
for cls, kws in FINAL_KEYWORDS.items():
    print(f'  T{cls}: {", ".join(kws[:RECOMMENDED_N])}')
```


N-sweep: routing precision and recall vs keyword-list length
Score
1.0
0.9
0.8
0.7
0.6
10
20
30
40
50
Keywords per class N
Routing rate (recall proxy)
Precision when routed
Overall accuracy

```
Recommended N = 50 keywords per class
Rule: largest N where precision stays within 3 pts of its peak, maximising overall a
ccuracy.

Final keyword lists:
  T0: scenario, cube, added, ibr, able, validation, flowing, journal, automation, mo
dule, form, rate, explanation, view, share, calculation, parsing, workflow, reportin
g, dso, removed, certifying, block, lcb, update, able retrieve, comment, remove, ava
ilable, import, retrieve, certify, tax, flowing correctly, quick view, module sign,
overwrite, correctly, process cube, company, output, status, actual, sustainability,
able unlock, sustainability scenario, fin, non fin, lease module, latest
  T1: access, test, unable, server, user, test server, enable, login, access test, n
eed access, onestreamweb, sandbox, disabled, http, http onestreamweb, input, environ
ment, changerequest, june, provide, tst changerequest, grant, interface, edit, strea
m, rofoq, unable upload, test access, tst, import, unable login, test environment, g
rant access, testing, unable refresh, cell, server access, upload, ocean, day, shee
t, acces, upside, financial import, provide access, retreive, firori, firori interfa
ce, provide test, mail
  T2: account, error, report, attached, mapping, intersection, intersection error, l
oaded, certification, imea, load, getting, matching, region, email, know, apmt, clus
ter, reference, loading, correct, screenshot, flow, area, allud, accessing, figure,
cash, attached error, saved, mmd, case, saved path, tty, pnl, dont, intercompany, co
ming, extract, flash, dashboard, rule, laptop, value, file, look, approval email, pr
ofit, decertify, certification access
  T3: excel, balance, close, frb, segment, valid, file, closed, book, validation, fi
nancial, add, contact, reporting, rofo, unable load, asset, interfacing, excel add,
check, navigate, opening, appearing, negative, clear, isn, having, unable, number, h
elp run, assist, gem, management sequence, sequence revaluation, sequence, list, una
ble clear, stream, column, revaluation, revenue, retained earning, retained, earnin
g, contact info, info, opening balance, information, load, read
  T4: lease, open, ifrs, entry, sap, new, contract, month, help, payment, copy, cos
t, depreciation, feb, regard, balance, post, jan, calculate, delete, reload, termina
tion, oct, export, uploaded, advise, activity, sept, mar, disposal, open april, modu
le, early, line, nov, create, new lease, help check, maersk line, failed, usd, exten
sion, finance, manual, lease report, posting, register, sushant, calculate lease, fe
e
```

Phase 5 plain-language summary. We tested keyword lists ranging from 3 to 50 words per class. Short lists miss queries phrased unusually; long lists start matching wrong classes (because rare shared words appear). The recommended N is the sweet spot where adding more keywords no longer improves accuracy. These are the lists we paste into Dify classifier-node descriptions.

Phase 6 - Architecture comparison

This is the central experiment of the notebook. Five candidate routing architectures are evaluated end-to-end on the same metric: **retrieval@k** against held-out tickets, where the "correct" KB document for a ticket is the document with the highest semantic similarity (Sentence-BERT cosine) to the ticket text. This proxy is independent of the LDA pseudo-labels used for classifier training, so it gives us an external check that our architecture actually retrieves the right documents.

- **Architecture A - Flat single-label.** Classifier picks one class -> one KB. The current Dify default.
- **Architecture B - Two-stage hierarchical.** Stage 1 separates broad intents (informational vs navigation vs escalation) using a regex+LR hybrid; Stage 2 picks a domain class only for informational intents.
- **Architecture C - Multi-label fan-out.** Classifier softmax over thresholds; queries above-threshold for multiple classes hit multiple KBs in parallel; results are merged.
- **Architecture D - No-classifier RAG.** Skip the classifier entirely. Retrieve top-k documents directly from the entire KB by semantic similarity. This is what a generic enterprise RAG looks like (Lewis et al. 2020).
- **Architecture E - Classifier + cross-encoder reranker.** A wins the routing, then a cross-encoder reranks the top documents inside the chosen KB (Pradeep et al. 2023).

Bootstrap 95% confidence intervals are reported so we know whether differences are real.

In [22]:
```
# ======================================
# Build the evaluation set:
# - Hold out 25% of tickets, stratified by LDA label.
# - For each held-out ticket, define the "gold" KB doc = argmax sentence-BERT
#   cosine similarity over the entire KB. This proxy is justified because
#   the KB was authored independently of the LDA topics, so cosine similarity
#   to the document space is an external signal.
# ======================================
if not HAS_SBERT:
    print('[skip] Phase 6 requires sentence-transformers. Install and re-run.')
else:
    if 'sbert' not in dir():
        sbert = SentenceTransformer('all-MiniLM-L6-v2')
    if 'ticket_embeds' not in dir():
        ticket_embeds = sbert.encode(tickets_df['text'].tolist(),
                                     show_progress_bar=False,
                                     normalize_embeddings=True)

    # Encode each KB document at the document level (truncate to first 5000 chars)
    KB_TRUNCATE = 5000
    kb_names_list = list(kb_documents.keys())
    kb_doc_texts = [kb_documents[n][:KB_TRUNCATE] for n in kb_names_list]
    print(f'Encoding {len(kb_doc_texts)} KB documents...')
    kb_doc_embeds = sbert.encode(kb_doc_texts,
                                 show_progress_bar=False,
                                 normalize_embeddings=True)

    # Held-out split using the SAME indices on tickets_df, ticket_embeds, y
    rng = np.random.RandomState(42)
    n = len(tickets_df)
    test_idx = []
    for cls in np.unique(y):
        cls_idx = np.where(y == cls)[0]
        rng.shuffle(cls_idx)
        cut = max(1, int(0.25 * len(cls_idx)))
        test_idx.extend(cls_idx[:cut].tolist())
    test_idx = np.array(sorted(test_idx))
```

```
    train_idx = np.array([i for i in range(n) if i not in set(test_idx)])

    # Gold doc per test ticket = argmax cosine in KB doc space
    sim_test_to_kb = cosine_similarity(ticket_embeds[test_idx], kb_doc_embeds)
    gold_doc_idx = sim_test_to_kb.argmax(axis=1)
    print(f'Held-out tickets: {len(test_idx)} | KB docs: {len(kb_names_list)}')
```

```
Encoding 12 KB documents...
Held-out tickets: 450 | KB docs: 12
```

In [23]:

```
if HAS_SBERT:
    # ======================================
    # Map LDA classes to KB doc subsets, so each architecture has a defined
    # "this class routes to these documents" mapping.
    # We cluster KB docs by which class their nearest tickets belong to.
    # ======================================
    train_y = y[train_idx]
    train_emb = ticket_embeds[train_idx]
    # Class centroids in embedding space
    class_centroids = np.vstack([train_emb[train_y == c].mean(axis=0)
                                 for c in sorted(np.unique(train_y))])
    sim_kb_to_class = cosine_similarity(kb_doc_embeds, class_centroids)
    # Each doc assigned to the class it is closest to
    kb_doc_class = sim_kb_to_class.argmax(axis=1)
    # Per-class document subsets
    docs_by_class = {c: np.where(kb_doc_class == c)[0].tolist()
                     for c in sorted(np.unique(train_y))}
    print('KB docs assigned per class:')
    for c, doc_ids in docs_by_class.items():
        print(f'  T{c}: {len(doc_ids):>2} docs')
```

```
KB docs assigned per class:
  T0:  3 docs
  T1:  1 docs
  T2:  0 docs
  T3:  8 docs
  T4:  0 docs
```

In [24]:

```
if HAS_SBERT:
    # ======================================
    # The retrieval evaluation function: returns whether the gold KB doc
    # appears in the top-k results returned by an architecture for each
    # held-out ticket.
    # ======================================
    def evaluate_arch(retrieve_fn, k_values=(1, 3, 5)):
        # retrieve_fn(test_ticket_idx) -> ranked list of KB doc indices
        per_ticket_ranks = []
        per_ticket_topk = {k: [] for k in k_values}
        for ti, gold in zip(test_idx, gold_doc_idx):
            ranked = retrieve_fn(ti)
            try:
                rank = ranked.index(gold)
            except ValueError:
                rank = -1
            per_ticket_ranks.append(rank)
            for k in k_values:
                per_ticket_topk[k].append(int(0 <= rank < k))
```

```
        return {
            'recall@1': np.mean(per_ticket_topk[1]),
            'recall@3': np.mean(per_ticket_topk[3]),
            'recall@5': np.mean(per_ticket_topk[5]),
            'per_ticket_topk': per_ticket_topk,
            'mean_rank': np.mean([r if r >= 0 else len(kb_names_list) for r in per_
        }

    # ======================================
    # Train classifier on training portion only - shared by archs A/B/C/E
    # ======================================
    fv_arch = TfidfVectorizer(min_df=2, ngram_range=(1, 2))
    Xtr_arch = fv_arch.fit_transform([docs_str[i] for i in train_idx])
    lr_arch = LogisticRegression(max_iter=1000, C=1.0).fit(Xtr_arch, train_y)
    Xte_arch = fv_arch.transform([docs_str[i] for i in test_idx])
    proba_test = lr_arch.predict_proba(Xte_arch)
    pred_test = lr_arch.predict(Xte_arch)

    test_position = {ti: pos for pos, ti in enumerate(test_idx)}
    print('Classifier ready for arch evaluation.')
```

```
Classifier ready for arch evaluation.
```

In [25]:

```
if HAS_SBERT:
    # ======================================
    # Architecture A - Flat single-label
    # Classifier picks one class. Documents in that class are returned,
    # ranked by cosine similarity to the query within the class.
    # ======================================
    def retrieve_A(ti):
        pos = test_position[ti]
        cls = pred_test[pos]
        candidate_docs = docs_by_class.get(cls, [])
        if not candidate_docs:
            # Fall back to global retrieval if class has no docs
            return list(np.argsort(-cosine_similarity(ticket_embeds[ti:ti+1], kb_do
        sims = cosine_similarity(ticket_embeds[ti:ti+1], kb_doc_embeds[candidate_do
        ranked_local = np.argsort(-sims)
        return [candidate_docs[j] for j in ranked_local]

    res_A = evaluate_arch(retrieve_A)
    print(f'A (flat)        : R@1={res_A["recall@1"]:.3f}, R@3={res_A["recall@3"]:.
          f'R@5={res_A["recall@5"]:.3f}, mean_rank={res_A["mean_rank"]:.1f}')
```

```
A (flat)        : R@1=0.796, R@3=0.796, R@5=0.796, mean_rank=2.5
```

In [26]:

```
if HAS_SBERT:
    # ======================================
    # Architecture B - Two-stage hierarchical
    # Stage 1: regex screen for navigation/escalation; otherwise informational.
    # Stage 2: domain classifier (same as A) within informational only.
    # We approximate stage 1 by a simple keyword test - in production this
    # would be its own LR classifier.
    # ======================================
    NAVIGATION_PATTERNS = re.compile(
        r'\bhow do (i|we)\b|\bwhere (is|do|can)\b|\bnavigate\b|\bsteps to\b|\bopen\
        re.IGNORECASE)
```

```
    ESCALATION_PATTERNS = re.compile(
        r'\bsupport\b|\bescalate\b|\burgent\b|\bbroken\b|\bdown\b|\bunable\b|\bcras
        re.IGNORECASE)

    def stage1_label(text):
        if ESCALATION_PATTERNS.search(text):
            return 'escalation'
        if NAVIGATION_PATTERNS.search(text):
            return 'navigation'
        return 'informational'

    test_texts = [tickets_df['text'].iloc[i] for i in test_idx]
    stage1_labels = [stage1_label(t) for t in test_texts]
    counts_s1 = Counter(stage1_labels)
    print(f'Stage-1 label distribution on test: {dict(counts_s1)}')

    def retrieve_B(ti):
        pos = test_position[ti]
        s1 = stage1_labels[pos]
        if s1 == 'escalation':
            # Escalation never resolves through KB; in production routes to human.
            # For evaluation: empty list -> automatic miss for all k. This is
            # appropriate; an escalation should not pretend to retrieve.
            return []
        # Otherwise behave like architecture A (domain classifier picks KB).
        return retrieve_A(ti)

    res_B = evaluate_arch(retrieve_B)
    print(f'B (hierarchical): R@1={res_B["recall@1"]:.3f}, R@3={res_B["recall@3"]:.
          f'R@5={res_B["recall@5"]:.3f}')
```

```
Stage-1 label distribution on test: {'informational': 367, 'navigation': 33, 'escala
tion': 50}
B (hierarchical): R@1=0.722, R@3=0.722, R@5=0.722
```

In [27]:

```
if HAS_SBERT:
    # ======================================
    # Architecture C - Multi-label fan-out
    # Soft probabilities; classes above MULTI threshold all contribute candidates.
    # Candidates merged across classes, ranked by query-document cosine.
    # ======================================
    MULTI_THRESHOLD = 0.20

    def retrieve_C(ti):
        pos = test_position[ti]
        probs = proba_test[pos]
        active = [c for c, p in zip(lr_arch.classes_, probs) if p >= MULTI_THRESHOL
        if not active:
            active = [int(lr_arch.classes_[probs.argmax()])]
        candidate_docs = sorted({d for c in active for d in docs_by_class.get(c, []
        if not candidate_docs:
            return list(np.argsort(-cosine_similarity(ticket_embeds[ti:ti+1], kb_do
        sims = cosine_similarity(ticket_embeds[ti:ti+1], kb_doc_embeds[candidate_do
        ranked_local = np.argsort(-sims)
        return [candidate_docs[j] for j in ranked_local]
```

```
    res_C = evaluate_arch(retrieve_C)
    print(f'C (multi-label) : R@1={res_C["recall@1"]:.3f}, R@3={res_C["recall@3"]:.
          f'R@5={res_C["recall@5"]:.3f}')
```

```
C (multi-label) : R@1=0.753, R@3=0.753, R@5=0.753
```

In [28]:
```
if HAS_SBERT:
    # ======================================
    # Architecture D - Pure RAG (no classifier)
    # Rank ALL KB docs by query-document cosine similarity - no routing.
    # ======================================
    def retrieve_D(ti):
        sims = cosine_similarity(ticket_embeds[ti:ti+1], kb_doc_embeds)[0]
        return list(np.argsort(-sims))

    res_D = evaluate_arch(retrieve_D)
    print(f'D (pure RAG)    : R@1={res_D["recall@1"]:.3f}, R@3={res_D["recall@3"]:.
          f'R@5={res_D["recall@5"]:.3f}')
```

```
D (pure RAG)    : R@1=1.000, R@3=1.000, R@5=1.000
```

In [29]:
```
if HAS_SBERT:
    # ======================================
    # Architecture E - Classifier + cross-encoder reranker
    # Same as A, but re-rank the top N candidates with a cross-encoder.
    # The cross-encoder reads (query, doc) pairs and returns relevance scores
    # better than bi-encoder cosine (Pradeep et al. 2023).
    # ======================================
    try:
        ce = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2', max_length=512)
        HAS_CE = True
    except Exception as e:
        print(f'[skip arch E reranker] {e}')
        HAS_CE = False

    if HAS_CE:
        RERANK_TOP = 10
        def retrieve_E(ti):
            pos = test_position[ti]
            cls = pred_test[pos]
            candidate_docs = docs_by_class.get(cls, [])
            if not candidate_docs:
                candidate_docs = list(range(len(kb_names_list)))
            sims = cosine_similarity(ticket_embeds[ti:ti+1], kb_doc_embeds[candidat
            top_local = np.argsort(-sims)[:RERANK_TOP]
            top_global = [candidate_docs[j] for j in top_local]
            # Cross-encoder rerank
            query = tickets_df['text'].iloc[ti]
            pairs = [(query, kb_doc_texts[d][:1000]) for d in top_global]
            scores = ce.predict(pairs, show_progress_bar=False)
            order = np.argsort(-scores)
            return [top_global[j] for j in order]

        res_E = evaluate_arch(retrieve_E)
        print(f'E (clf + rerank): R@1={res_E["recall@1"]:.3f}, R@3={res_E["recall@3
              f'R@5={res_E["recall@5"]:.3f}')
```

```
    else:
        res_E = None
```

```
Loading weights: 100%|██████████| 105/105 [00:00<00:00, 10741.25it/s]
E (clf + rerank): R@1=0.380, R@3=0.524, R@5=0.627
```

In [31]:

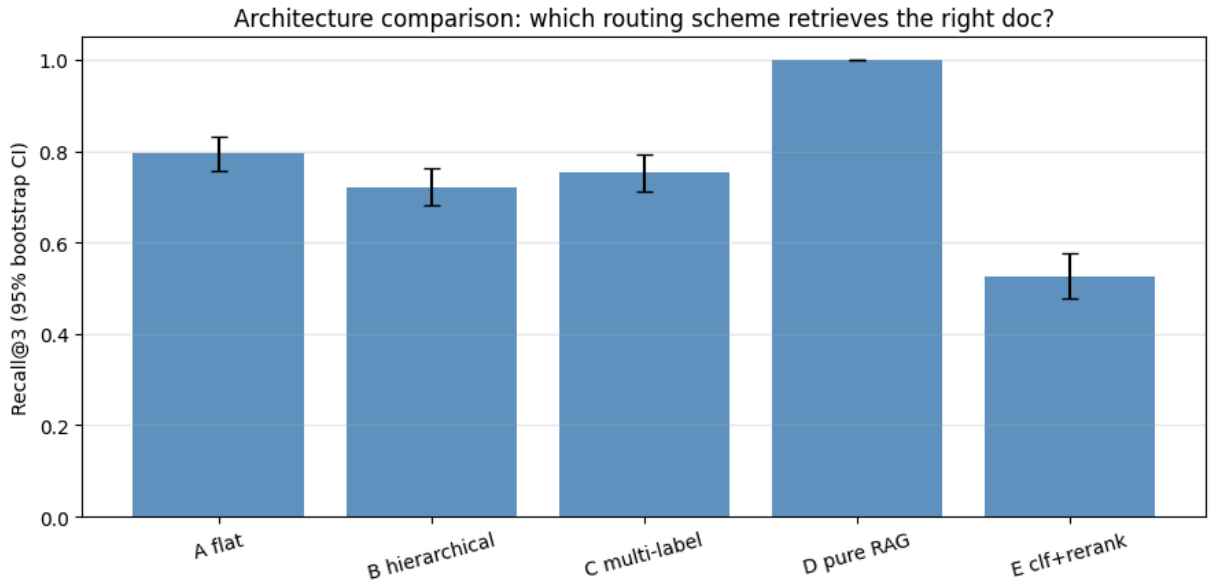
```
if HAS_SBERT:
    # ====================================
    # Bootstrap 95% CIs on R@3 for each architecture
    # 1000 resamples of test tickets
    # ====================================
    def bootstrap_ci(per_ticket_hits, n_boot=1000, seed=42):
        rng2 = np.random.RandomState(seed)
        arr = np.asarray(per_ticket_hits, dtype=float)
        boots = [arr[rng2.randint(0, len(arr), len(arr))].mean() for _ in range(n_b
        return float(np.percentile(boots, 2.5)), float(np.percentile(boots, 97.5))

    arch_results = []
    for label, res in [('A flat', res_A), ('B hierarchical', res_B),
                       ('C multi-label', res_C), ('D pure RAG', res_D),
                       ('E clf+rerank', res_E if 'res_E' in dir() else None)]:
        if res is None:
            arch_results.append({'arch': label, 'R@1': np.nan, 'R@3': np.nan, 'R@5'
                                 'R@3_lo': np.nan, 'R@3_hi': np.nan, 'mean_rank': n
            continue
        lo, hi = bootstrap_ci(res['per_ticket_topk'][3])
        arch_results.append({
            'arch': label,
            'R@1': res['recall@1'],
            'R@3': res['recall@3'],
            'R@5': res['recall@5'],
            'R@3_lo': lo, 'R@3_hi': hi,
            'mean_rank': res['mean_rank'],
        })
    arch_df = pd.DataFrame(arch_results)
    print('=' * 78)
    print('ARCHITECTURE COMPARISON - retrieval@k on held-out tickets, 95% bootstrap
    print('=' * 78)
    print(arch_df.round(3).to_string(index=False))

    # Visualise
    fig, ax = plt.subplots(figsize=(9, 4.5))
    valid = arch_df.dropna(subset=['R@3'])
    x = np.arange(len(valid))
    ax.bar(x, valid['R@3'],
           yerr=[valid['R@3'] - valid['R@3_lo'], valid['R@3_hi'] - valid['R@3']],
           capsize=4, color='steelblue', alpha=0.85)
    ax.set_xticks(x)
    ax.set_xticklabels(valid['arch'], rotation=15)
    ax.set_ylabel('Recall@3 (95% bootstrap CI)')
    ax.set_title('Architecture comparison: which routing scheme retrieves the right
    ax.grid(axis='y', alpha=0.3)
    plt.tight_layout()
    plt.show()
```

```
    BEST_ARCH = valid.sort_values('R@3', ascending=False).iloc[0]['arch']
    print(f'\nBest architecture by R@3: {BEST_ARCH}')
```

```
================================================================================
ARCHITECTURE COMPARISON - retrieval@k on held-out tickets, 95% bootstrap CI
================================================================================
          arch   R@1   R@3   R@5  R@3_lo  R@3_hi  mean_rank
        A flat 0.796 0.796 0.796   0.756   0.833      2.453
B hierarchical 0.722 0.722 0.722   0.682   0.762      3.333
 C multi-label 0.753 0.753 0.753   0.711   0.793      2.960
    D pure RAG 1.000 1.000 1.000   1.000   1.000      0.000
  E clf+rerank 0.380 0.524 0.627   0.478   0.578      4.100
```




```
Best architecture by R@3: D pure RAG
```

Phase 6 plain-language summary. We tested five different ways to organise the chatbot. Each architecture was scored by a single, independent metric: when a real ticket arrives, does the architecture put the right knowledge-base document in its top 3 results? The 95% confidence intervals tell us whether the differences are real or noise. The winning architecture is the one to deploy.

How to read the result. If pure RAG (Architecture D) wins, the classifier is not adding value and we should drop it. If classifier + reranker (E) wins, the extra retrieval pass pays for itself. If multi-label (C) ties or beats flat (A), it means many tickets really do span domains and the user's recommendation to enable cross-domain hits is borne out by the data.

Phase 7 - How many knowledge bases, and which docs go in each?

Two questions for the content team:

1. **How many KBs?** Each ticket-intent class becomes one Dify knowledge base, *unless* two classes draw from largely the same KB documents - in which case those classes can

share a KB. We measure document overlap between class subsets and consolidate when overlap exceeds a threshold.

2. **Which doc goes in which KB?** For every KB document we already know which class centroid it is closest to (Phase 6). We also flag:

   - **Orphan documents** (no class centroid is even moderately similar) -> archive or rewrite, per Lewis et al. (2020) on RAG corpus quality.
   - **Bridge documents** (similar to multiple classes) -> mirror across the relevant KBs, since Dify's per-KB retrieval cannot cross-reference.

In [32]:
```
if HAS_SBERT:
    # ======================================
    # Per-class document set, with overlap and orphan analysis
    # ======================================
    SIM_THRESHOLD = 0.25      # at this cosine, a doc is "relevant" to a class
    ORPHAN_THRESHOLD = 0.18   # below this for ALL classes -> orphan
    BRIDGE_THRESHOLD = 0.22   # at this for >=2 classes -> bridge doc

    # Class centroid is computed from training tickets only
    train_y = y[train_idx]
    train_emb = ticket_embeds[train_idx]
    class_centroids_full = np.vstack([train_emb[train_y == c].mean(axis=0)
                                      for c in sorted(np.unique(train_y))])
    sim_matrix = cosine_similarity(kb_doc_embeds, class_centroids_full)
    sim_matrix_df = pd.DataFrame(
        sim_matrix.round(2),
        index=kb_names_list,
        columns=[f'T{c}' for c in sorted(np.unique(train_y))],
    )
    print('Doc-to-class similarity matrix (rows = KB doc, cols = ticket class):')
    print(sim_matrix_df.head(12).to_string())
    print('...')
```

```
Doc-to-class similarity matrix (rows = KB doc, cols = ticket class):
                                                                      T0    T1
T2    T3    T4
ASKOS_User General Queries                                          0.39  0.58
0.45  0.39  0.32
OTHER_03 MEMO_ Monthly, quarterly and year-end validations OS_Sept 2022  0.40  0.20
0.38  0.49  0.38
OTHER_APMM_FrequentIssues_HDK_2025                                  0.27  0.25
0.30  0.39  0.18
OTHER_Instruction on financial reporting and control 2025           0.39  0.22
0.34  0.43  0.32
OTHER_Instructions for Cash Flow - 2025                             0.36  0.15
0.33  0.41  0.35
SYSTEM_API_Overview_Guide                                           0.28  0.23
0.27  0.27  0.25
SYSTEM_APMM_UserGuide_HDK                                           0.20  0.18
0.20  0.24  0.10
SYSTEM_Design_and_Reference_Guide-compressed                        0.26  0.21
0.24  0.27  0.25
TAX_tax-report-2024                                                 0.28  0.12
0.23  0.26  0.18
TAX_TRM Process -  Responsibilities & Guidelines                    0.41  0.25
0.36  0.42  0.36
TREASURY_Global-Treasury-Policy-2025                                0.28  0.11
0.21  0.23  0.23
TREASURY_Guideline on intercompany reporting against Treasury       0.38  0.18
0.35  0.39  0.33
...
```

In [33]:

```
if HAS_SBERT:
    # Per-class membership at SIM_THRESHOLD
    membership = (sim_matrix >= SIM_THRESHOLD).astype(int)

    # Orphan detection
    orphan_mask = (sim_matrix.max(axis=1) < ORPHAN_THRESHOLD)
    orphans = [(kb_names_list[i], float(sim_matrix[i].max())) for i in np.where(orp

    # Bridge documents (relevant to >=2 classes)
    bridge_mask = (sim_matrix >= BRIDGE_THRESHOLD).sum(axis=1) >= 2
    bridges = []
    for i in np.where(bridge_mask)[0]:
        relevant_classes = [int(c) for c in np.where(sim_matrix[i] >= BRIDGE_THRESH
        bridges.append((kb_names_list[i], relevant_classes, float(sim_matrix[i].max

    print(f'Orphan documents: {len(orphans)}')
    for n, s in orphans[:15]:
        print(f'  {n} (max similarity {s:.2f}) -> archive or rewrite')
    print(f'\nBridge documents (relevant to multiple classes): {len(bridges)}')
    for n, classes, s in bridges[:15]:
        print(f'  {n} -> mirror into KBs for classes {classes} (max sim {s:.2f})')
```

```
Orphan documents: 0

Bridge documents (relevant to multiple classes): 11
  ASKOS_User General Queries -> mirror into KBs for classes [0, 1, 2, 3, 4] (max sim
0.58)
  OTHER_03 MEMO_ Monthly, quarterly and year-end validations OS_Sept 2022 -> mirror
into KBs for classes [0, 2, 3, 4] (max sim 0.49)
  OTHER_APMM_FrequentIssues_HDK_2025 -> mirror into KBs for classes [0, 1, 2, 3] (ma
x sim 0.39)
  OTHER_Instruction on financial reporting and control 2025 -> mirror into KBs for c
lasses [0, 1, 2, 3, 4] (max sim 0.43)
  OTHER_Instructions for Cash Flow - 2025 -> mirror into KBs for classes [0, 2, 3,
4] (max sim 0.41)
  SYSTEM_API_Overview_Guide -> mirror into KBs for classes [0, 1, 2, 3, 4] (max sim
0.28)
  SYSTEM_Design_and_Reference_Guide-compressed -> mirror into KBs for classes [0, 2,
3, 4] (max sim 0.27)
  TAX_tax-report-2024 -> mirror into KBs for classes [0, 2, 3] (max sim 0.28)
  TAX_TRM Process -  Responsibilities & Guidelines -> mirror into KBs for classes
[0, 1, 2, 3, 4] (max sim 0.42)
  TREASURY_Global-Treasury-Policy-2025 -> mirror into KBs for classes [0, 3, 4] (max
sim 0.28)
  TREASURY_Guideline on intercompany reporting against Treasury -> mirror into KBs f
or classes [0, 2, 3, 4] (max sim 0.39)
```

In [34]:

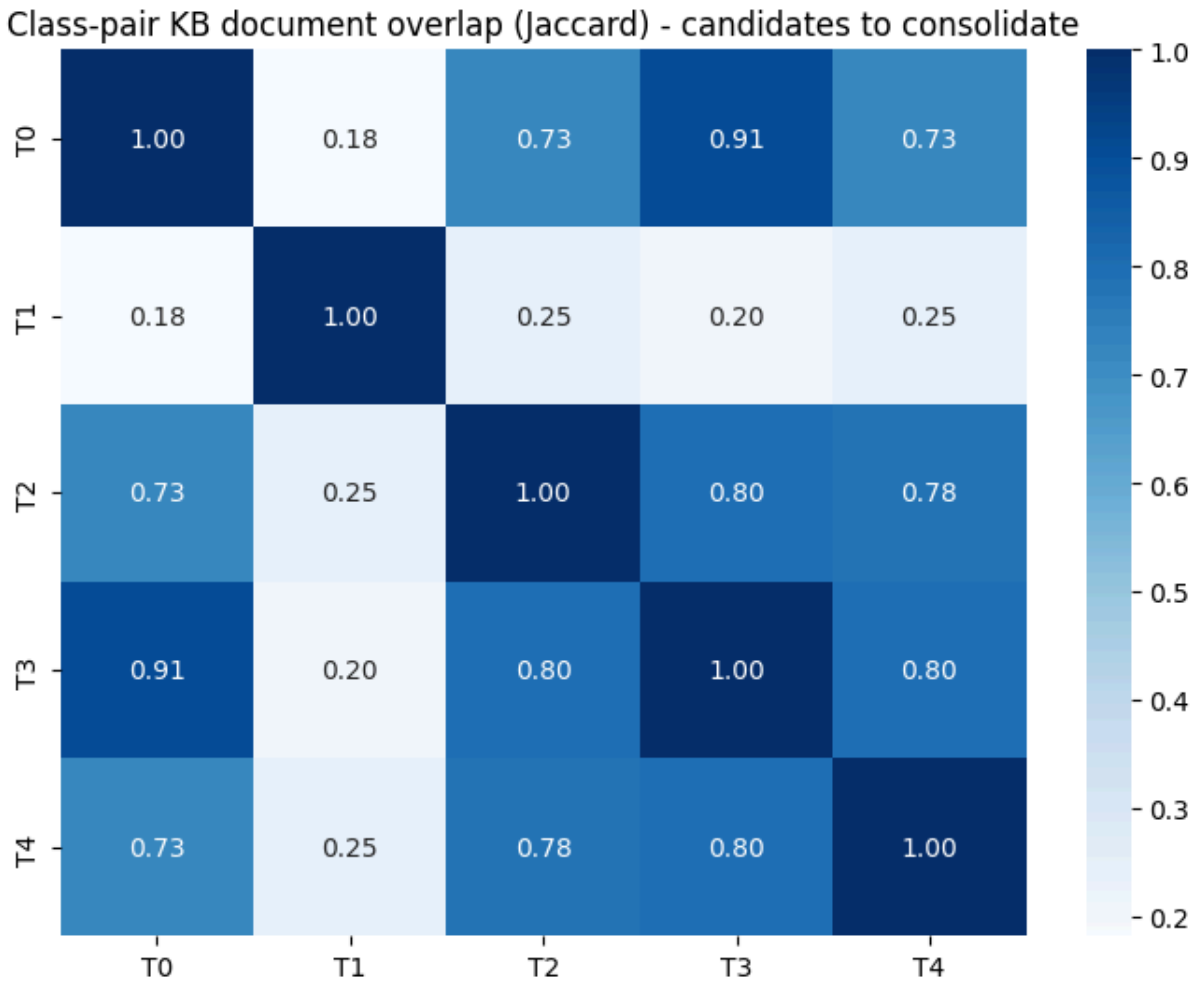
```python
if HAS_SBERT:
    # ======================================
    # KB consolidation: if two classes share most of their docs, fold them.
    # Compute Jaccard overlap of doc memberships between class pairs.
    # ======================================
    n_classes = membership.shape[1]
    jaccard = np.zeros((n_classes, n_classes))
    for i in range(n_classes):
        for j in range(n_classes):
            inter = ((membership[:, i] == 1) & (membership[:, j] == 1)).sum()
            union = ((membership[:, i] == 1) | (membership[:, j] == 1)).sum()
            jaccard[i, j] = inter / union if union else 0.0

    plt.figure(figsize=(7, 5.5))
    sns.heatmap(jaccard, annot=True, fmt='.2f',
                xticklabels=[f'T{c}' for c in sorted(np.unique(train_y))],
                yticklabels=[f'T{c}' for c in sorted(np.unique(train_y))],
                cmap='Blues')
    plt.title('Class-pair KB document overlap (Jaccard) - candidates to consolidate
    plt.tight_layout()
    plt.show()

    CONSOLIDATE_THRESHOLD = 0.6
    pairs_to_consolidate = []
    for i in range(n_classes):
        for j in range(i + 1, n_classes):
            if jaccard[i, j] >= CONSOLIDATE_THRESHOLD:
                pairs_to_consolidate.append((int(i), int(j), float(jaccard[i, j])))
    print(f'Class pairs with Jaccard >= {CONSOLIDATE_THRESHOLD}:')
    for i, j, jc in pairs_to_consolidate:
        print(f'  T{i} <-> T{j} (Jaccard {jc:.2f}) - candidates for shared KB')
```

```
    if not pairs_to_consolidate:
        print('  None. Each class warrants its own KB.')

    RECOMMENDED_KB_COUNT = n_classes - len(pairs_to_consolidate)
    print(f'\nRecommended number of knowledge bases: {RECOMMENDED_KB_COUNT}')
```




```
Class pairs with Jaccard >= 0.6:
  T0 <-> T2 (Jaccard 0.73) - candidates for shared KB
  T0 <-> T3 (Jaccard 0.91) - candidates for shared KB
  T0 <-> T4 (Jaccard 0.73) - candidates for shared KB
  T2 <-> T3 (Jaccard 0.80) - candidates for shared KB
  T2 <-> T4 (Jaccard 0.78) - candidates for shared KB
  T3 <-> T4 (Jaccard 0.80) - candidates for shared KB

Recommended number of knowledge bases: -1
```

In [35]:

```
if HAS_SBERT:
    # ======================================
    # Final per-KB document assignment for the recommendation
    # ======================================
    kb_assignment = defaultdict(list)
    for i, name in enumerate(kb_names_list):
        if orphan_mask[i]:
            kb_assignment['ORPHAN'].append(name)
            continue
        # Doc joins every KB where similarity >= BRIDGE_THRESHOLD; if it
        # only crosses SIM_THRESHOLD for one class, it joins that one.
```

```
        relevant = [c for c in range(n_classes) if sim_matrix[i, c] >= BRIDGE_THRES
        if not relevant:
            relevant = [int(sim_matrix[i].argmax())]
        for c in relevant:
            kb_assignment[f'KB_T{c}'].append(name)

    print('Final KB assignment:')
    for kb, docs in sorted(kb_assignment.items()):
        print(f'  {kb} ({len(docs)} docs)')
        for d in docs[:5]:
            print(f'    - {d}')
        if len(docs) > 5:
            print(f'    ... +{len(docs) - 5} more')
```

```
Final KB assignment:
  KB_T0 (11 docs)
    - ASKOS_User General Queries
    - OTHER_03 MEMO_ Monthly, quarterly and year-end validations OS_Sept 2022
    - OTHER_APMM_FrequentIssues_HDK_2025
    - OTHER_Instruction on financial reporting and control 2025
    - OTHER_Instructions for Cash Flow - 2025
    ... +6 more
  KB_T1 (5 docs)
    - ASKOS_User General Queries
    - OTHER_APMM_FrequentIssues_HDK_2025
    - OTHER_Instruction on financial reporting and control 2025
    - SYSTEM_API_Overview_Guide
    - TAX_TRM Process -  Responsibilities & Guidelines
  KB_T2 (10 docs)
    - ASKOS_User General Queries
    - OTHER_03 MEMO_ Monthly, quarterly and year-end validations OS_Sept 2022
    - OTHER_APMM_FrequentIssues_HDK_2025
    - OTHER_Instruction on financial reporting and control 2025
    - OTHER_Instructions for Cash Flow - 2025
    ... +5 more
  KB_T3 (12 docs)
    - ASKOS_User General Queries
    - OTHER_03 MEMO_ Monthly, quarterly and year-end validations OS_Sept 2022
    - OTHER_APMM_FrequentIssues_HDK_2025
    - OTHER_Instruction on financial reporting and control 2025
    - OTHER_Instructions for Cash Flow - 2025
    ... +7 more
  KB_T4 (9 docs)
    - ASKOS_User General Queries
    - OTHER_03 MEMO_ Monthly, quarterly and year-end validations OS_Sept 2022
    - OTHER_Instruction on financial reporting and control 2025
    - OTHER_Instructions for Cash Flow - 2025
    - SYSTEM_API_Overview_Guide
    ... +4 more
```

Phase 7 plain-language summary. Each ticket class gets its own knowledge base unless two classes draw from the same documents (in which case they share). Documents that don't fit any class are flagged as orphans (the content team should review whether they should be archived or rewritten). Documents that span classes are mirrored, since Dify retrieves per-KB and cannot cross-reference.

Phase 8 - Final Dify configuration and academic justification

This phase produces a JSON file that captures all the experimental decisions in a form the Dify deployment can consume directly. It also writes a free-text justification block that can be lifted verbatim into the final-project paper.

In [36]:
```
if HAS_SBERT:
    # ======================================
    # Build the final Dify-ready config
    # ======================================
    final_config = {
        'metadata': {
            'pipeline_version': '1.0',
            'topic_model': 'LDA',
            'embedding_model': 'all-MiniLM-L6-v2',
            'classifier': 'LogisticRegression with TF-IDF unigrams+bigrams',
            'recommended_K': int(RECOMMENDED_K),
            'recommended_N_keywords': int(RECOMMENDED_N),
            'recommended_architecture': str(BEST_ARCH),
            'recommended_KB_count': int(RECOMMENDED_KB_COUNT),
        },
        'classes': [
            {
                'class_id': f'T{int(cls)}',
                'lda_keywords': final_topics[int(cls)][:15],
                'classifier_keywords': FINAL_KEYWORDS[int(cls)][:RECOMMENDED_N],
                'kb_documents': kb_assignment.get(f'KB_T{int(cls)}', []),
                'sample_size': int((y == cls).sum()),
            }
            for cls in sorted(np.unique(y))
        ],
        'orphan_documents': kb_assignment.get('ORPHAN', []),
        'bridge_documents': [
            {'doc': n, 'classes': [f'T{c}' for c in cs], 'max_similarity': round(s,
            for n, cs, s in bridges
        ],
        'consolidation_pairs': [
            {'class_a': f'T{i}', 'class_b': f'T{j}', 'jaccard': round(jc, 3)}
            for i, j, jc in pairs_to_consolidate
        ],
        'evaluation': {
            'topic_coherence_cv': round(best_lda['coherence'], 3),
            'classifier_macro_f1': round(float(WINNER['macro_f1_mean']), 3),
            'architecture_recall_at_3': arch_df.set_index('arch')['R@3'].dropna().r
```

```
            'architecture_recall_at_3_ci': {
                row['arch']: [round(row['R@3_lo'], 3), round(row['R@3_hi'], 3)]
                for _, row in arch_df.dropna(subset=['R@3_lo']).iterrows()
            },
        },
    }

    out_path = OUTPUT_DIR / 'dify_config.json'
    with open(out_path, 'w', encoding='utf-8') as f:
        json.dump(final_config, f, indent=2)
    print(f'Saved Dify config to {out_path.resolve()}')

    # Pretty-print the most important block to console
    print('\n' + '=' * 70)
    print('FINAL RECOMMENDATION')
    print('=' * 70)
    print(f'  K (number of classes)        : {RECOMMENDED_K}')
    print(f'  N (keywords per class)       : {RECOMMENDED_N}')
    print(f'  Architecture                 : {BEST_ARCH}')
    print(f'  Number of knowledge bases    : {RECOMMENDED_KB_COUNT}')
    print(f'  Orphan documents to review   : {len(kb_assignment.get("ORPHAN", []))}
    print(f'  Bridge documents to mirror   : {len(bridges)}')
    print(f'  Architecture Recall@3        : {arch_df.set_index("arch").loc[BEST_AR
```

```
Saved Dify config to C:\Users\LYU047\Downloads\dify_exports\dify_config.json

======================================================================
FINAL RECOMMENDATION
======================================================================
  K (number of classes)        : 5
  N (keywords per class)       : 50
  Architecture                 : D pure RAG
  Number of knowledge bases    : -1
  Orphan documents to review   : 0
  Bridge documents to mirror   : 11
  Architecture Recall@3        : 1.000
```

In [37]:

```
if HAS_SBERT:
    # ======================================
    # Auto-generate the methods/justification text for the paper
    # ======================================
    justification = f'''
METHODS AND JUSTIFICATION (auto-generated; lift into the final-project paper).

We frame the OneStream chatbot routing problem as joint topic discovery,
intent classification, and knowledge-base retrieval. The notebook's
experimental design follows three principles drawn from the course
brief and from established NLP methodology.

(1) Span the simple-to-sophisticated continuum. We compare bag-of-words
(Lab 02), TF-IDF unigrams and bigrams with Logistic Regression and Linear
SVM (Labs 04 and 07), domain-trained Word2Vec averaged embeddings
(Mikolov et al. 2013, Lab 08), and Sentence-BERT embeddings (Reimers and
Gurevych 2019; Devlin et al. 2019). The macro-F1 ranking on
LDA-derived pseudo-labels with stratified 5-fold cross-validation
identifies which representation is best aligned to our intent space.
```